

VIA University College

23/10/1944 - Procesrapport

Nikolaj Bræmer Christensen – 354322

Jonathan Cilius Østberg Winther Engell Gøtze – 354500

Victor Bruun Fassbender – 354361

PELC

Antal tegn:

XR Softwareingeniøruddannelsen

4. semester

27. maj 2026

Indhold

1	Indledning	2
2	Gruppearbejde	4
3	Projektinitiering	6
4	Projektudførelse	8
4.1	Oprettelse af projekt	8
4.2	Game Sessions	9
4.3	TeamSelection	10
4.4	VoiceChat	11
4.5	Kundekontakt & 3D-skanning af Katedralskolen	13
4.6	Timer	14
4.7	Class Selection	16
4.8	Specielle Evner	17
4.9	nøgler og kisten	22
4.10	Fange Mechanic	23
4.11	befri fængslet	24
4.12	Win Conditions	25
4.13	map opsætning	26
4.14	AI NPC'er	27
5	Personlige Refleksioner	28
5.1	Jonathan	28
5.2	Victor	29
5.3	Nikolaj	30
6	Refleksion over vejledning	31
7	Konklusion	32

1 Indledning

I anledning af, at Viborg Katedralskole fejrer bygningens 100-års jubilæum, valgte gruppen at udvikle et spil, der både fejrer og hylder skolens historie, arkitektur og kulturelle betydning. Gruppen ønskede at skabe et projekt, som kombinerer moderne teknologi med historisk formidling, og som samtidig kunne give brugeren en interaktiv oplevelse af skolens omgivelser og historie. Motivation for projektet opstod ud fra et ønske om at arbejde med et lokalt og meningsfuldt emne, hvor der kunne skabes en tæt forbindelse mellem det tekniske produkt og den historiske fortælling.

I projektets opstartsfasen arbejdede gruppen med idéudvikling og research omkring skolens historie og arkitektoniske udtryk. Gennem møder med skolens historielærere og gennemgang af historisk materiale blev gruppen introduceret til skolens rolle under Anden Verdenskrig. Her blev det blandt andet beskrevet, hvordan modstandsfolk benyttede skolen til produktion og distribution af illegale aviser under den tyske besættelse. Gruppen fandt denne fortælling særligt interessant, da den både havde historisk betydning og samtidig kunne omsættes til spændende gameplay-elementer et digitalt spil.

På baggrund af dette valgte gruppen at udvikle et online multiplayer-spil, hvor spillerne enten kan spille som modstandsfolk eller som nazister. Modstandsfolkene skal bevæge sig rundt på skolen og udføre forskellige opgaver, såsom at Finde Nøgler som refere til Skolens Logo og Historie eller undgå at blive opdaget, mens den modsatte side forsøger at finde og stoppe dem. Gruppen ønskede gennem dette gameplay at skabe spænding og samarbejde mellem spillerne, samtidig med at spillet tog inspiration fra virkelige historiske begivenheder.

En vigtig del af projektet var at skabe et visuelt realistisk miljø, som kunne give spillerne følelsen af faktisk at befinde sig på skolen. Derfor blev spillet udviklet i Unreal Engine, da gruppen vurderede, at denne engine gav de bedste muligheder for høj grafisk kvalitet, realistisk lysopsætning og detaljerede omgivelser. Gruppen havde desuden et ønske om at lære mere om professionel spiludvikling og arbejde med

værktøjer, der anvendes i den moderne spilindustri.

For at gøre omgivelserne så virkelighedstro som muligt anvendte gruppen 3D-scanninger af skolens bygninger og områder. Dette blev gjort ved hjælp af programmet RealityScan, som gjorde det muligt at omdanne billeder af bygningerne til detaljerede 3D-modeller såkaldt photogrammetry. Disse modeller blev efterfølgende importeret til spillet og viderebearbejdet, så de kunne anvendes i det digitale miljø. Gruppen oplevede, at dette både gav en mere autentisk oplevelse og samtidig gjorde projektet mere teknisk udfordrende, da arbejdet med 3D-scanning og optimering krævede nye kompetencer inden for modellering og performanceoptimering.

Projektforløbet blev organiseret ud fra en Scrum-lignende arbejdsmetode med sprint-baseret udvikling. Gruppen valgte denne agile tilgang for at skabe en fleksibel arbejdsproces, hvor der løbende kunne arbejdes med planlægning, udvikling, test og evaluering. Hvert sprint havde fokus på bestemte funktioner eller dele af spillet, hvilket gjorde det muligt at arbejde mere struktureret og samtidig tilpasse projektet undervejs. Gruppen anvendte blandt andet møder og opgavefordeling, for at sikre overblik over arbejdsprocessen og kommunikationen mellem gruppemedlemmerne.

Igennem projektet oplevede gruppen både tekniske og samarbejds-mæssige udfordringer. Særligt arbejdet med multiplayer-funktioner, 3D-modeller og koordinering mellem forskellige arbejdsområder krævede løbende problemløsning og samarbejde. Samtidig gav projektet gruppen mulighed for at udvikle kompetencer inden for programmering, projektstyring, kommunikation og kreativ problemløsning. Gruppen fik desuden erfaring med, hvordan historiske fortællinger kan integreres i digitale medier for at skabe en engagerende brugeroplevelse.

Procesrapporten vil derfor beskrive projektets udviklingsforløb, arbejdsmetoder og samarbejde samt gruppens refleksioner over læringsudbytte, udfordringer og erfaringer gennem hele projektperioden.

2 Gruppearbejde

Gruppen har som tidligere nævnt arbejdet ud fra en Scrum-lignende proces gennem hele projektforsløbet. I denne struktur har gruppemedlemmerne haft forskellige roller, som både har været baseret på personlige styrker og på deres samarbejdsevner.

Nikolaj har fungeret som gruppens Scrum Master gennem hele projektet. Denne rolle har passet godt til hans strukturelle tilgang til arbejde samt hans erfaring med planlægning og ledelse fra tidligere ophold i forsvaret. Han har haft fokus på at sikre overblik over arbejdsprocessen, holde styr på sprintforløb og understøtte gruppens fremdrift. Derudover har han bidraget til at skabe struktur i arbejdet og sikre, at opgaver blev fordelt og fulgt op på.

Victor har haft rollen som Product Owner. Denne rolle er blevet tildelt ham på baggrund af hans stærke kommunikative evner og hans evne til at formidle idéer klart til både gruppen og eksterne parter. Victor har desuden stået for den primære kommunikation med projektets "kunde", hvilket har været vigtigt for at sikre, at projektets retning og krav blev forstået og opdateret løbende gennem forløbet. Hans rolle har derfor været central i forhold til at afstemme forventninger og sikre, at projektet bevægede sig i den ønskede retning.

Jonathan har fungeret som gruppens analytiske specialist. Han har bidraget med velovervejede analyser, refleksioner og faglige vurderinger i både udviklingen af spillet og i rapportskrivningen. Hans tilgang til arbejdet har i høj grad været præget af en analytisk og struktureret tankegang, som stemmer overens med hans blå DISC-personprofil. Dette har været en styrke i forhold til at sikre kvalitet i gruppens beslutninger og skabe et solidt grundlag for de tekniske og designmæssige valg i projektet.

Gruppen har arbejdet i et Scrum-lignende system, hvor der blandt andet er blevet anvendt Planning Poker til estimering af use cases og tildeling af story points. Dette har hjulpet gruppen med at prioritere opgaver og skabe et bedre overblik over arbejdsbyrden i de enkelte sprint. Projektet har været opdelt i korte sprintforløb, typisk

bestående af intensive arbejdsperioder fra mandag til onsdag, og så fra onsdag til fredag samt et længere “weekend-sprint” fra fredag til mandag. Denne struktur gjorde det muligt at arbejde fokuseret i afgrænsede perioder og løbende evaluere fremdriften.

Hvert sprint blev afsluttet med et retrospektiv og et review-møde. Under retrospektivet blev det gennemførte arbejde gennemgået, og der blev “brændt” story points for færdiggjorte opgaver, hvilket gav gruppen et overblik over fremdriften og arbejdsindsatsen. I review-fasen blev arbejdsprocessen, samarbejdet og de anvendte metoder diskuteret, så gruppen løbende kunne justere og forbedre sin arbejdsgang. Dette har været en vigtig del af den iterative udvikling, da det har givet mulighed for kontinuerlig forbedring af både struktur og samarbejde.

Ved sprint planning blev backloggen gennemgået, og nye use cases blev prioriteret og fordelt mellem gruppemedlemmerne. Dette sikrede en klar forventningsafstemning i begyndelsen af hvert sprint og gav en fælles forståelse af, hvilke mål der skulle opnås.

Gruppen har dog ikke anvendt fuld Scrum-metodologi, da der eksempelvis ikke blev afholdt daglige stand-up møder. Dette var et bevidst valg, da gruppen ønskede en lettere og mere fleksibel tilgang til metoden, som bedre passede til projektets størrelse og arbejdsform. Derudover vurderede gruppen, at den løbende kommunikation internt var tilstrækkelig til at håndtere eventuelle udfordringer uden faste daglige møder. Da sprintene samtidig var relativt korte, oplevede gruppen, at der stadig var en god kontinuerlig forståelse for projektets status uden behov for daglige Scrum-møder.

Samlet set har denne arbejdsform bidraget til en struktureret, men fleksibel proces, hvor roller, planlægning og løbende evaluering har været centrale elementer i gruppens samarbejde.

3 Projektinitiering

Gruppen valgte at holde problemfelt og problemformulering mere åbne for at undgå at definere den endelige løsning for tidligt i projektet. Dette var inspireret af PBL-tilgangen, hvor problemfeltet skal beskrive konteksten uden at skabe tunnelsyn omkring en bestemt løsning eller historisk retning.

Gruppen valgte efterfølgende at tage udgangspunkt i razziaen fra oktober 1943, da denne begivenhed repræsenterede en markant og dramatisk del af skolens historie under besættelsen. Begivenheden gav samtidig mulighed for at kombinere historisk formidling med multiplayer gameplay og samarbejdsmechanikker i Unreal Engine 5. Projektets historiske tema førte naturligt til flere etiske overvejelser. Gruppen diskuterede tidligt i projektet, hvordan historiske roller skulle repræsenteres i spillet. Selvom projektet var inspireret af virkelige hændelser fra besættelsen, var målet ikke at skabe en fuldstændig historisk simulation, men derimod en multiplayeroplevelse inspireret af skolens historie. Der opstod derfor refleksioner omkring balancen mellem historisk autenticitet og spildesign. Gruppen valgte blandt andet, at spillere ikke skulle fremstilles som elever i nazistiske roller. Denne beslutning blev taget for at undgå en u hensigtsmæssig identifikation mellem nuværende elever og historiske aktører fra besættelsen. Samtidig blev flere gameplay-elementer, såsom abilities og movement-mekanikker, designet ud fra etablerede multiplayergenrer fremfor historisk realisme. Dette gjorde det muligt at prioritere et engagerende gameplay-loop, samtidig med at spillet stadig bevarede en historisk inspiration.

Gruppen anvendte primært en Scrum-Lignende metode som overordnet arbejdsproces. Projektet blev opdelt i forskellige sprints, hvor der først blev arbejdet med idéudvikling og planlægning, hvorefter implementation og test blev udført løbende, denne tilgang blev valgt da flere medlemmer i gruppen, tidligere har syntes at et Fuldt Scrum forløb, er for meget Process og det kan være hæmende for at få udviklet noget, omvendt har medlemmerne også oplevet at en meget løser process så som Kanban, er for lidt process også, derfor blev der fundet et kompromis, ved at tage dele fra Scrum, scrum møde, retrospective, rewive, samt sprint planing og planig poker, dog afholdets der ikke dagligscrum meetings, da det vurderes at havde medlemmer brug for hjælp til udviklingen af deres gældende case, så ville de selvstændigt kommunikere det ud

til gruppen, dette blev også valgt da sprint længden var sat til kun 2 dage, for at stadig havde en vis rytme og hastighed i processen. Kommunikationen foregik hovedsageligt gennem Discord, hvor gruppen afholdt online møder cirka hver anden dag. Dette gjorde det muligt at koordinere arbejdsopgaver og følge projektets fremdrift.

4 Projektudførelse

4.1 Oprettelse af projekt

For at komme i gang med projektet oprettede gruppen først selve Unreal Engine-projektet. Projektet blev initialt oprettet som et C++-projekt baseret på en First Person-template, da gruppen ønskede, at spillet skulle opleves fra et førstepersonsperspektiv. Dette valg blev truffet tidligt i processen for at sikre, at gameplay-oplevelsen blev defineret allerede fra starten, hvilket senere har haft betydning for designvalg og udvikling af spilmekanikker.

Efter oprettelsen af projektet blev der opsat et fælles GitHub-repository for at understøtte samarbejdet i gruppen. Dette gjorde det muligt at arbejde parallelt på projektet, samtidig med at der blev skabt en struktureret versionstyring af filer og ændringer. Brugen af pull requests og code reviews blev implementeret for at sikre kvalitet i koden og for at give mulighed for faglig sparring mellem gruppemedlemmerne. Denne arbejdsform viste sig at være vigtig for at undgå konflikter i koden og for at sikre, at ændringer blev gennemgået og forstået af flere i gruppen, før de blev integreret i hovedprojektet.

Tidligt i udviklingsfasen blev der eksperimenteret med Unreal Engine, særligt i forhold til samspillet mellem Blueprints og C++. Gruppen undersøgte, hvordan Blueprints kunne anvendes til hurtig prototyping af gameplay-elementer, mens C++ blev brugt til mere komplekse og performancekritiske systemer. Denne kombination viste sig at være central for projektets udvikling, da den gav en god balance mellem fleksibilitet og kontrol.

I begyndelsen af projektet bar arbejdet præg af eksperimentering og læring, hvor gruppen brugte tid på at opbygge forståelse for engine'ens struktur og arbejdsmetoder. Dette betød, at fremdriften i starten var relativt langsom, men til gengæld lagde det et solidt fundament for den videre udvikling. Efterhånden som gruppen fik større teknisk forståelse, blev arbejdsprocessen mere effektiv, og udviklingen af spillets funktioner kunne ske mere struktureret og målrettet.

4.2 Game Sessions

den første del af systemet som blev udviklet var Faktisk Online Session systemet, dette blev gjort, for at sikre at alle efterfølgende systemer kunne testes i en online kontekst, og for at sikre at vi havde en stabil base at bygge videre på. Dette system blev udviklet ved hjælp af Unreal Engine's Online Subsystem, som giver en række værktøjer og funktioner til at håndtere online multiplayer-funktionalitet. Implementeringen omfattede oprettelse og håndtering af sessions, matchmaking, og netværskommunikation mellem klienter og servere. Dette var en vigtig milepæl i projektet, da det lagde grundlaget for alle de multiplayer-aspekter, der senere skulle implementeres i spillet.

i praksis blev implementationen af online sessions først oprettet som et system hvor spilleren kunne oprette en session, og spiller der joined ville altid senere

4.3 TeamSelection

4.4 VoiceChat

Ved den indledende undersøgelse af, hvad der lå bag et funktionelt "proximity" voicechat-system, var den generelle konsensus online, at det var bedst at betale \$40 og spare hovedpinen. Eksisterende YouTube-guides gjorde oftest brug af et tredjepartsplugin, "Advanced Sessions", som eksponerer flere funktioner og attributter fra Unreal Engines Online Subsystem.

Dette havde til formål at muliggøre brugen af Steams version af Online Subsystem, en spilplatform udviklet af Valve. Herigennem ville det være muligt at gøre brug af "Steam Voice Chat", som håndterer hele logikken bag VoIP (Voice over Internet Protocol).

Denne løsning krævede dog, at projektet som helhed skiftede til brug af Steams Online Subsystem, hvilket samtidig medførte behov for refaktorering af andre aspekter af spillet. Ved nærmere undersøgelse af systemet rapporterede flere brugere også problemer med tick rate og båndbredde, og derfor var skiftet ikke særligt attraktivt.

I den indledende udvikling af voicechat-systemet blev det derfor forsøgt at følge disse guides, men med udskiftning af funktionalitet, der var afhængig af Steams systemer, til tilsvarende funktioner fra Unreals eget subsystem. Komplexiteten ved dette viste sig dog at være for høj og førte til mange timers arbejde med at forsøge at tilpasse funktioner til noget, de ikke oprindeligt var udviklet til.

Problemet lå især i, at Unreal Engines subsystem ikke kan håndtere VoIP uden eksterne plugins, hvilket "Advanced Sessions" forsøgte at løse. Kommunikation mellem de to systemer krævede ændringer i projektets engine-konfigurationsfiler, specifikt i filen DefaultEngine.ini. Ved brug af kommandoerne "[OnlineSubsystem] bHasVoiceEnabled=true" og "[Voice] bEnabled=true" blev VoIP-funktionaliteten eksponeret til "Advanced Sessions".

Herefter kunne der oprettes en funktion, som skaber et objekt af typen UVOIPTalker ved oprettelsen af player characters. Denne UVOIPTalker bliver herefter bundet til hver enkelt spiller og muliggør brugen af VoIP.

Som standard fungerer voicechat på global skala, hvilket betyder, at der ikke differentieres mellem spillernes lydniveau afhængigt af afstand, og at spillere altid kan høre hinanden uanset afstand. Dette blev løst ved at modificere UVOIPTalkers "Voice Settings", hvor der ved brug af Sound Attenuation blev tilføjet en maksimal

afstand på hver spillers lydoutput.

Ved at kombinere Sound Attenuation med systemet for team selection og de tilhørende tags, som hver spiller modtager, blev der desuden skabt to separate Sound Attenuations unikke for hvert hold. Modstandsbevægelsens Sound Attenuation benytter standardværdier, mens Gestapos har forhøjet volume, reverb og max distance. Dette betyder, at de kan høres tydeligere, med mere rumklang og fra længere afstande.

Til sidst blev et Widget Blueprint udviklet til at vise spillerens nuværende mikrofoninput ved runtime. Her anvendes en progress bar, som modtager et input mellem 0 og 1 og visualiserer den aktuelle værdi med en grøn bjælke.

Værdien mellem 0 og 1 blev skabt ved at tage et Audio Envelope Value, som under test varierede mellem -0,7 og -2,7 afhængigt af lydniveauet. Denne værdi blev herefter clamped til intervallet mellem 0 og 1 og sendt videre til progress baren.

4.5 Kundekontakt & 3D-skanning af Katedralskolen

Kundekontakt foregik primært gennem mailkorrespondance med Simon Justesen og Signe Bro, som er lærere ansat på Katedralskolen. Derudover blev der sendt mails til tre elever fra skolen med henblik på potentiel musikproduktion til projektet.

Yderligere kommunikation og koordinering fandt sted i forbindelse med skanningen af skolen, hvor det var muligt at planlægge det kommende samarbejdsforløb sammen med Simon Justesen, som deltog under skanningen.

Skanningen af skolen blev udført i samarbejde med den anden gruppe, som ligeledes havde valgt Katedralskolen som projekt. I forbindelse med dette blev der lånt udstyr fra Klok Visuals. Skanningen begyndte mandag d. 20. april kl. 17.00.

De skannede lokaler omfattede gangarealer, klasseværelser, laboratorier, gymnastiksal, bibliotek, musiklokale, balsal samt gårdspladsen.

4.6 Timer

Formålet med opgaven var både at skabe en fungerende countdown-timer til gameplayet og samtidig opnå en bedre forståelse af Unreal Engines multiplayer-arkitektur, herunder kommunikationen mellem server og klient samt samspillet mellem GameMode, GameState og PlayerController.

Da projektet allerede indeholdt en eksisterende MainUI-widget, tog jeg udgangspunkt i denne. Jeg tilføjede et tekst-element til brugerfladen, som blev gemt som variabelen `RemainingTime`. Under arbejdet fandt jeg frem til, at MainUI-widgetten bliver oprettet i PlayerController og derefter tilføjet til spillerens skærm.

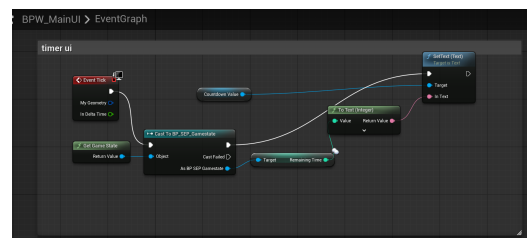
For at få timeren til at fungere korrekt i multiplayer blev variabelen `RemainingTime` placeret i GameState som en replicated variabel. Dette gjorde det muligt for alle klienter at modtage den samme opdaterede tid fra serveren. Selve countdown-logikken blev implementeret i GameMode ved hjælp af Unreal Engines "Set Timer By Event"-node.

I GameMode oprettede jeg to custom events: `Start Countdown` og `Countdown Event`. `Countdown Event` blev sat som delegate til "Set Timer By Event" med looping aktiveret. Eventet hentede derefter den aktuelle GameState, castede til GameState, hentede værdien af `RemainingTime`, reducerede værdien med ét sekund og opdaterede derefter variabelen igen.

Denne opgave var udfordrende, da jeg i starten havde begrænset erfaring med Unreal Engines multiplayer-struktur og replicated variabler. Jeg brugte derfor en stor del af tiden på research og testning for at forstå, hvordan de forskellige dele af systemet kommunikerer. Gennem processen fik jeg en væsentligt bedre forståelse af

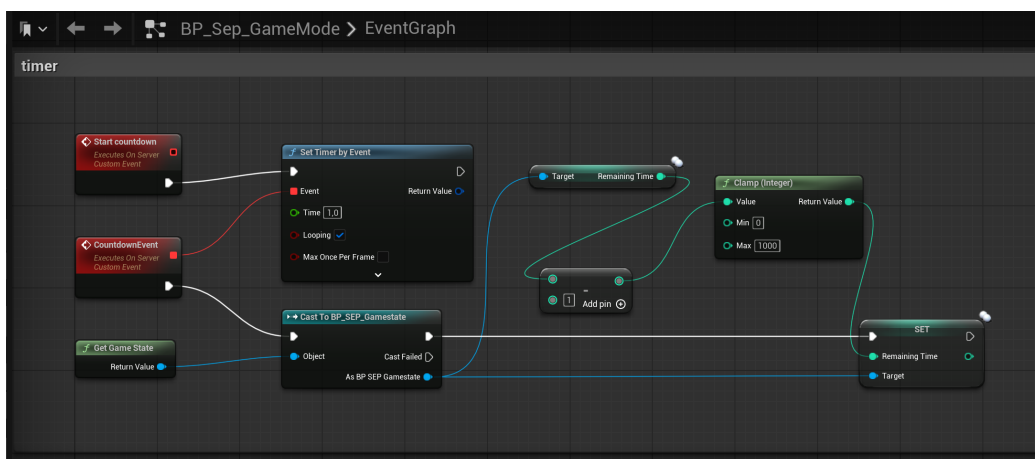


Figur 1: Oprettelse af MainUI i PlayerController.



Figur 2: Timer UI-elementet RemainingTime i MainUI.

server-authoritative systemer i Unreal Engine og hvordan GameMode og GameState anvendes forskelligt i multiplayer-spil.



Figur 3: Countdown-logik implementeret i GameMode.

4.7 Class Selection

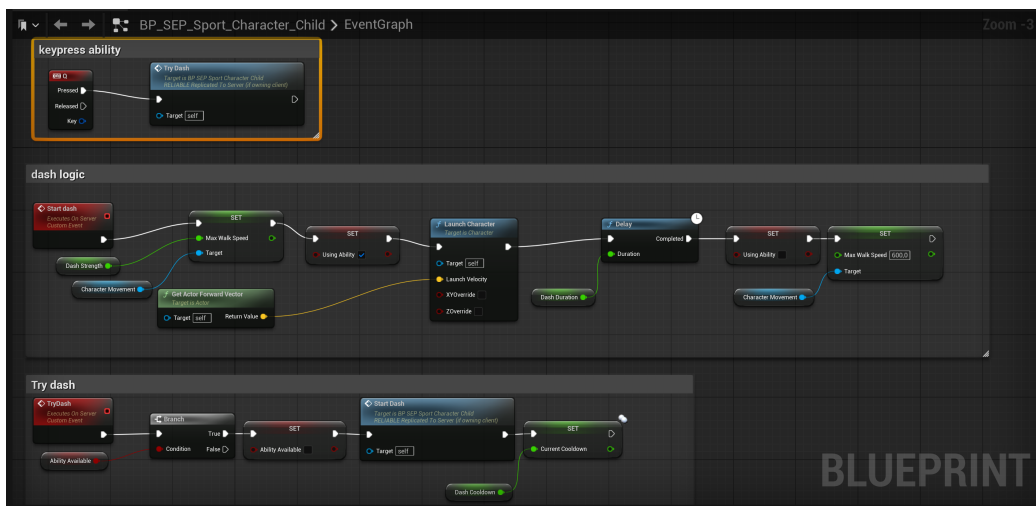
4.8 Specielle Evner

En af de største tekniske udfordringer i projektet var implementeringen af abilities til de forskellige robber-karakterer. Tidligt i projektforløbet besluttede gruppen, at spillet skulle indeholde forskellige robber-typer med unikke abilities. De planlagte karakterer var blandt andet Athlete, Scientist, Language og Music robber. På dette tidspunkt var kun karakterernes overordnede temaer fastlagt, mens de konkrete abilities endnu ikke var designet.

Jeg begyndte arbejdet med abilities ved først at implementere en simpel dash-ability til Athlete-karakteren. Formålet var at skabe en relativt simpel mechanic, som samtidig kunne hjælpe mig med at forstå, hvordan abilities bedst kunne struktureres i Unreal Engine Blueprints.

Ability-systemet for dash blev opdelt i flere custom events, blandt andet **TryDash**, **StartDash** og inputhåndtering via keypress-events. I **TryDash** blev der først kontrolleret, om abilityen var tilgængelig gennem variabelen **AbilityAvailable**. Hvis abilityen kunne anvendes, blev variabelen sat til **false**, cooldown blev startet, og **StartDash**-eventet blev kaldt.

I **StartDash** blev karakterens bevægelseshastighed midlertidigt ændret ved hjælp af **CharacterMovement**-komponenten. Derudover blev **Launch Character**-funktionen



Figur 4: Dash-ability logik for Athlete-karakteren.

anvendt sammen med `GetActorForwardVector` for at sikre, at dash-bevægelsen fulgte den retning, spilleren kiggede i. Efter en kort delay-periode svarende til abilityens varighed blev karakterens normale hastighed gendannet.

Cooldown-systemet blev implementeret ved hjælp af variablen `CurrentCooldown`, som kontinuerligt blev reduceret med `Delta Seconds` på hvert `Event Tick`. Da cooldowns er spiller-specifikke data, fandt jeg undervejs ud af, at disse værdier ikke burde placeres i `GameState` eller `GameMode`, men i stedet håndteres lokalt i den enkelte karakterklasse. Dette var en vigtig læring i forhold til Unreal Engines multiplayer-arkitektur.

Udvikling af UI til abilities

Efter implementeringen af de første abilities begyndte jeg at generalisere systemet og udvikle et fælles UI til cooldowns og ability-information. Hver ability indeholdt blandt andet variablerne:

- `CurrentCooldown` (float)
- `AbilityName` (string)
- `AbilityAvailable` (bool)

En af de største udfordringer var kommunikationen mellem ability-klasserne og UI-systemet. Jeg havde tidligere arbejdet med replicated værdier i `GameState` til den globale spil-timer, men cooldowns skulle i stedet opdateres lokalt hos den enkelte spiller.

Den løsning jeg endte med at implementere, var at gemme en reference til `MainUI`-widgetten i `PlayerController`. Når UI-widgetten blev oprettet med `Create Main UI Widget`, blev returværdien gemt som en objekt-reference kaldet `MainUIRef`.

Derefter kunne ability-klasserne på hvert `Event Tick` caste til `PlayerController` og opdatere UI'et gennem et custom event kaldet `AbilityUI` i `MainUI`-blueprintet. Eventet modtog blandt andet ability-navn og resterende cooldown som parametre og opdaterede derefter tekstfelterne i UI'et via `SetText`-funktioner.

Denne del af projektet var særligt udfordrende, da jeg skulle opnå en forståelse af, hvordan UI, `PlayerController` og gameplay-klasser kommunikerer i Unreal Engine.

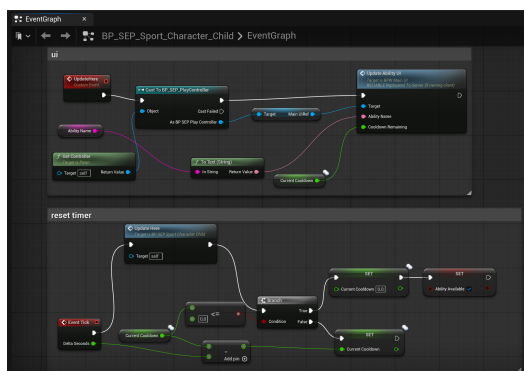
Gennem processen lærte jeg meget om referencehåndtering, objektkommunikation og forskellen mellem replicated data og lokale spillerdata i multiplayer-spil.

Efter implementeringen af dash-abilityen begyndte jeg at arbejde mere systematisk med abilities og deres tilhørende cooldown-system. På dette tidspunkt havde jeg opnået en bedre forståelse af Unreal Engines Blueprint-struktur og multiplayer-arkitektur, hvilket gjorde det muligt at generalisere flere dele af systemet. Figur 7 viser den overordnede struktur for stun-abilityen samt kommunikationen mellem gameplay-logik og UI.

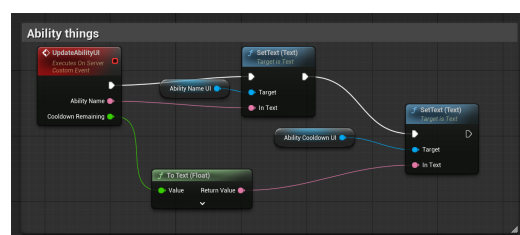
Stun-abilityen blev implementeret som en area-of-effect (AoE) ability, hvor spillere inden for en bestemt radius bliver påvirket midlertidigt. Abilityen blev opdelt i flere separate events for at gøre logikken mere overskuelig og lettere at debugge. De centrale events var blandt andet **TryStun**, **StartStun** samt forskellige events til håndtering af cooldown og UI-opdateringer.

Når spilleren trykker på ability-knappen, kaldes **TryStun**-eventet. Her bliver der først kontrolleret, om abilityen er tilgængelig ved hjælp af variabelen **AbilityAvailable**. Hvis abilityen er klar til brug, sættes variabelen til **false**, cooldown-værdien sættes til den definerede **StunCooldown**, og **StartStun**-eventet bliver aktiveret.

I **StartStun**-eventet anvendes funktionen **Get Overlapping Actors** til at finde spillere inden for abilityens radius. Herefter gennemløbes alle fundne aktører med et **For Each Loop**. For at sikre at kun modstanderholdet blev påvirket, blev der anvendt **Actor Has Tag** til at kontrollere, om karakteren havde det relevante tag.



Figur 5: Ability-klassens kommunikation med UI via PlayerController.



Figur 6: Opdatering af ability-tekst i MainUI.

Hvis betingelsen var opfyldt, blev karakteren castet til den relevante character-klasse.

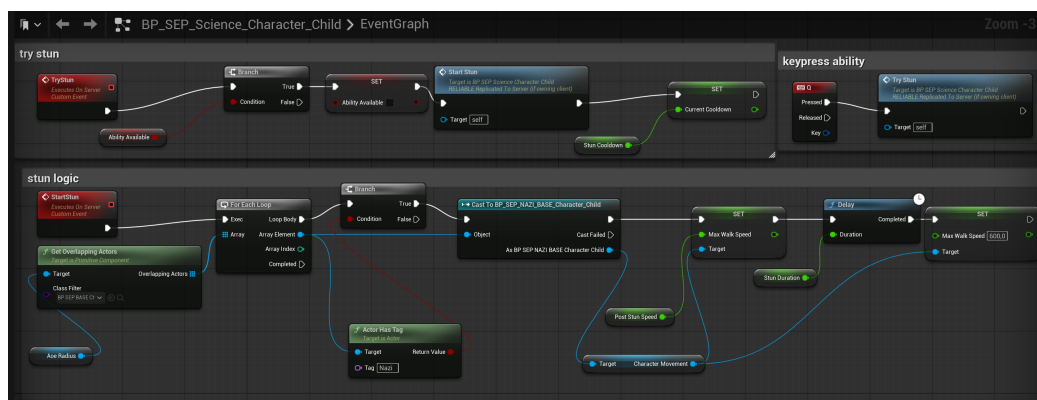
Selve stun-effekten blev implementeret ved midlertidigt at ændre spillerens **Max Walk Speed** gennem **CharacterMovement**-komponenten. Når abilityen aktiveres, reduceres spillerens bevægelseshastighed kraftigt, hvilket effektivt forhindrer normal bevægelse. Efter en delay-periode svarende til **StunDuration** blev hastigheden sat tilbage til standardværdien.

Cooldown-systemet blev håndteret lokalt i character-klassen gennem variablen **CurrentCooldown**. På hvert **Event Tick** blev cooldown-værdien reduceret med **Delta Seconds**, indtil værdien nåede nul. Når cooldown-værdien var nul eller lavere, blev **AbilityAvailable** sat tilbage til **true**, hvilket gjorde abilityen tilgængelig igen.

Udviklingen af UI til abilities viste sig at være mere kompleks end forventet. Da cooldowns er spiller-specifikke data, var det ikke hensigtsmæssigt at placere dem i **GameState** eller **GameMode**, som tidligere var blevet anvendt til den globale spil-timer. I stedet blev UI-opdateringen håndteret lokalt via **PlayerController**.

Da **MainUI**-widgetten oprettes i **PlayerController**, gemte jeg en reference til widgetten i variablen **MainUIRef**. Dette gjorde det muligt for character-klassen at caste til **PlayerController** og kalde custom eventet **UpdateAbilityUI**. Eventet modtog blandt andet ability-navn og resterende cooldown som parametre og opdaterede derefter tekstfelterne i UI'et ved hjælp af **SetText**-funktioner.

Denne del af projektet krævede omfattende debugging og research, da kommunikationen mellem gameplay-klasser og UI i Unreal Engine var mere kompleks end



Figur 7: Stun-ability logik for Scientist-karakteren.

forventet. Gennem processen opnåede jeg en bedre forståelse af objekt-referencer, lokal spillerdatahåndtering og forskellen mellem replicated data og klient-specifikke data i multiplayer-spil.

4.9 nøgler og kisten

4.10 Fange Mechanic

4.11 befri fængslet

4.12 Win Conditions

4.13 map opsætning

4.14 AI NPC'er

5 Personlige Refleksioner

5.1 Jonathan

Jeg tog en mindre rolle i projektets opstart og dette påvirkede mit engagement i de tidlige faser af projektet, og jeg oplevede også perioder, hvor jeg ikke deltog i møder eller informerede gruppen om fravær på forhånd. Set i bakspejlet kunne jeg have bidraget mere aktivt. Jeg tror, at min manglende deltagelse i starten kan have været en form for social loafing, hvor jeg ubevidst trak mig tilbage fra ansvaret, fordi jeg følte, at andre i gruppen ville tage sig af opgaverne, eller allerede havde taget sig af dem. For at bryde ud af denne dynamik forsøgte jeg at bidrage på områder, hvor jeg oplevede, at der manglede arbejdskraft, blandt andet gennem kildearbejde og rapportskrivning. Da jeg som person har en tendens til at være tilbageholdene i sociale sammenhænge, var dette en måde jeg stadig kunne bidrage på. Jeg oplevede at gruppen håndterede situationen professionelt og inkluderende, hvilket gjorde det lettere for mig at bidrage mere aktivt senere i projektforsløbet. Jeg har lært, at det er vigtigt at kommunikere åbent om ens deltagelse og forpligtelser i et gruppeprojekt for at sikre, at alle medlemmer føler sig ansvarlige og engagerede. I fremtidige projekter vil jeg stræbe efter at være mere proaktiv i min deltagelse og kommunikere klart med mine gruppemedlemmer om mine bidrag og eventuelle udfordringer, jeg måtte have.

5.2 Victor

I forhold til selve implementeringsfasen af projektet endte jeg med at tage en meget specifik vinkel på projektet, nemlig implementeringen af voicechat-systemet. Dette endte dog med at være det eneste, jeg bidrog med i forhold til implementering, grundet uforudsete komplikationer. Set i bakspejlet burde jeg have været bedre til at bede om hjælp til visse aspekter af arbejdet, således at jeg kunne have arbejdet på flere dele af spillet frem for kun én enkelt.

Jeg har til tider været sent ude med mine svar i gruppen og missede også et enkelt scrum-møde, hvilket er beklageligt. Især mod slutningen af projektet låste jeg mig fast på at få voicechat-systemet til at fungere, hvilket førte til forsinkede GitHub-pushes, da jeg havde et stort fokus på, at det system, jeg arbejdede på, skulle være fuldt funktionelt.

Den vigtigste læring, jeg tager med fra dette projekt, er især ikke at hyperfiksere på et bestemt problem, men i stedet at blive bedre til at sprøge om hjælp til- og fordele større arbejdsopgaver blandt gruppemedlemmer. På den måde kan jeg fremover få større indflydelse på flere dele af udviklingen og dermed også opnå et bedre indblik i projektets forskellige systemer.

5.3 Nikolaj

6 Refleksion over vejledning

Gruppen har oplevet, at vejledningen har fungeret godt i den del af forløbet, hvor der stadig har været planlagt undervisning og faste vejledningstimer. I denne periode har det været muligt at få støtte og sparring, hvilket har bidraget positivt til projektets fremdrift.

Samtidig har gruppen oplevet, at det kan være begrænsende kun at have en enkelt vejleder tilknyttet projektet, da vejlederen ikke nødvendigvis har haft faglig indsigt inden for alle relevante områder. Dette har i nogle tilfælde gjort det vanskeligt at få helt konkret og brugbar feedback på mere specialiserede problemstillinger, særligt i forhold til de Formaila aspekter af projektet.

Efter den planlagte undervisnings- og vejledningsperiode har gruppen desuden erfaret, at det kræver en mere proaktiv tilgang selv at tage kontakt til vejlederne ved opståede problemer eller tvivlsspørgsmål. Gruppen har i den forbindelse lært, at det er vigtigt aktivt at opsøge vejledning løbende, frem for at afvente faste møder.

Fremadrettet vil gruppen derfor fokusere på at være mere opsøgende og struktureret i brugen af vejledere, blandt andet ved at forberede konkrete spørgsmål på forhånd og tage kontakt tidligere i processen, når udfordringer opstår. Dette forventes at kunne forbedre udbyttet af vejledningen og bidrage til en mere effektiv projektudvikling.

7 Konklusion